

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3
«Создание таблиц базы данных PostgreSQL. Заполнение таблиц
рабочими данными» по дисциплине «Проектирование и реализация
баз данных»
Вариант свой. БД «Пассажир»**

Обучающийся Дидык Глеб Олегович
Факультет прикладной информатики
Группа K3239
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии
Преподаватель Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург

2026

1. Цель работы

Овладеть практическими навыками создания таблиц базы данных PostgreSQL, заполнения их рабочими данными, резервного копирования и восстановления базы данных.

2. Оборудование и программное обеспечение

Оборудование: компьютерный класс.

Программное обеспечение: СУБД PostgreSQL, pgAdmin 4.

3. Практическое задание

1. Создать базу данных с использованием pgAdmin 4 согласно индивидуальному заданию.
2. Создать схему в составе базы данных.
3. Создать таблицы базы данных.
4. Установить ограничения на данные: Primary Key, Unique, Check, Foreign Key.
5. Заполнить таблицы базы данных рабочими данными.
6. Создать резервные копии базы данных: Custom для восстановления и Plain SQL для листинга.
7. Восстановить базу данных из резервной копии.

4. Выполнение работы

4.1. Наименование базы данных и вариант

В рамках лабораторной работы создана база данных Passenger. Предметная область базы данных — учет пассажиров, паспортов, железнодорожных станций, поездов, вагонов, маршрутов, рейсов, билетных касс и проданных билетов.

Схема базы данных в PostgreSQL имеет имя passenger_schema. Имена таблиц, полей, ограничений и значения справочников выполнены на английском языке.

4.2. Инфологическая модель базы данных

Инфологическая модель базы данных разработана по предметной области «Passenger». Модель содержит сущности: passport, passenger, station, ticket_office, train, carriage, train_carriage, route, route_station, trip и ticket.

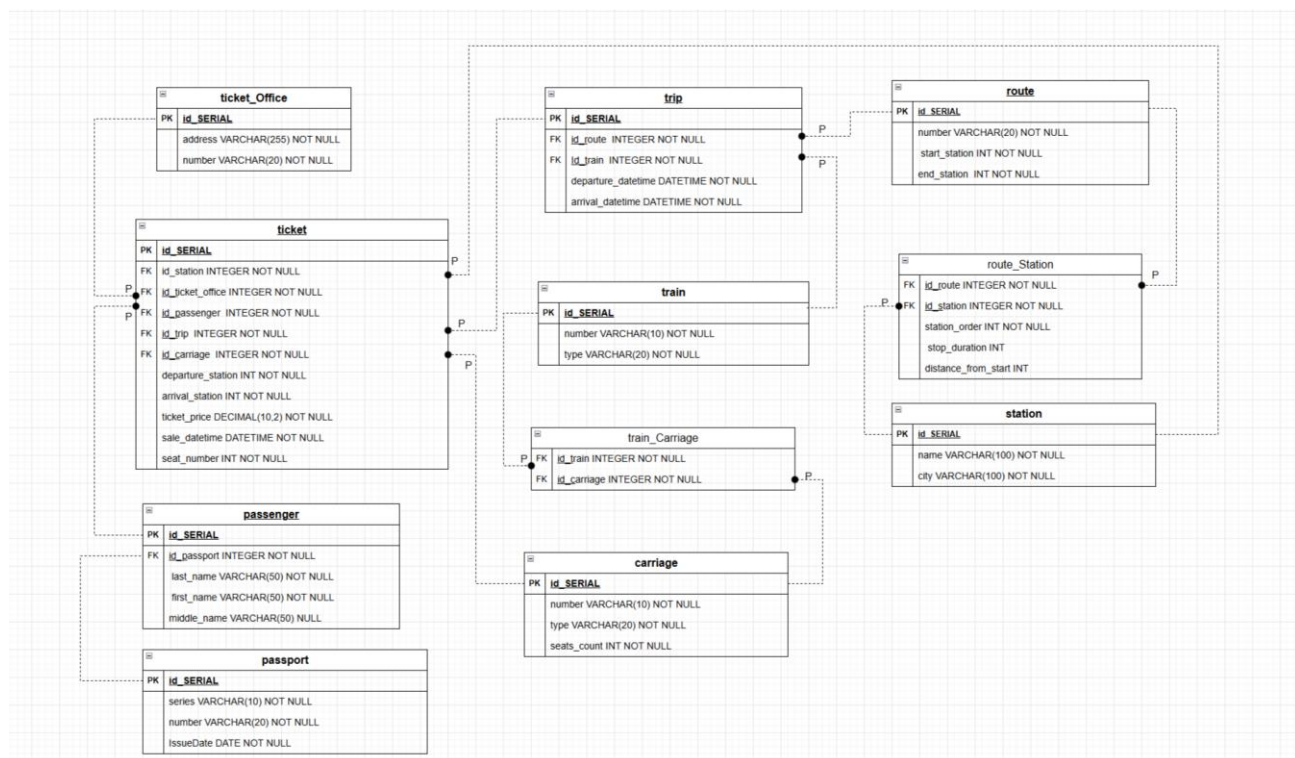


Рисунок 1 – Инфологическая модель базы данных

4.3. Логическая ERD-модель базы данных

Логическая модель базы данных сгенерирована средствами pgAdmin Generate ERD. В реализации базы данных используются простые суррогатные ключи id для основных сущностей. Для таблиц train_carriage и route_station используются составные первичные ключи, так как эти таблицы реализуют связи многие-ко-многим.

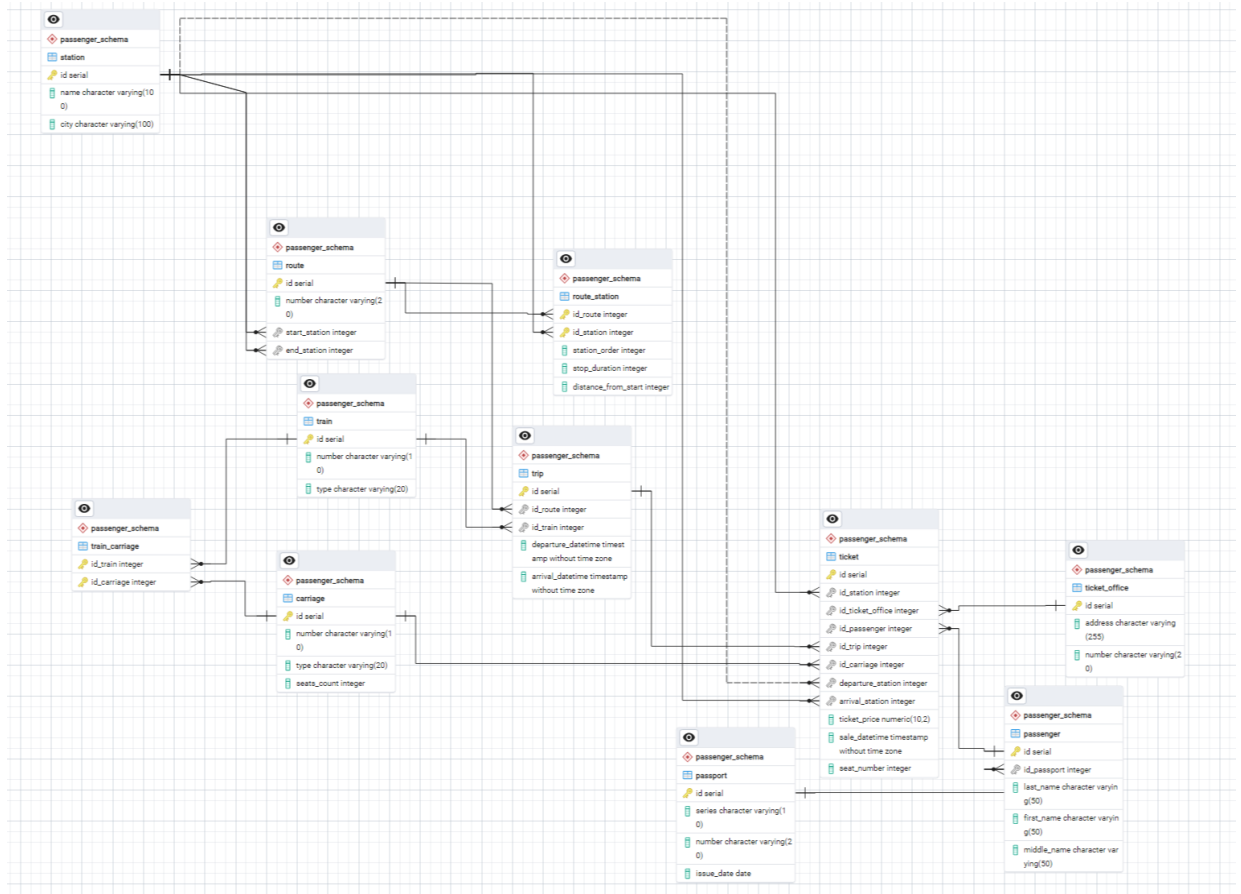


Рисунок 2 – Схема логической модели базы данных, сгенерированная в Generate ERD

4.4. Создание схемы, таблиц и ограничений

Ниже приведен SQL-код создания схемы, таблиц и ограничений базы данных. В коде используются первичные ключи, внешние ключи, ограничения уникальности и проверочные ограничения.

```
CREATE SCHEMA IF NOT EXISTS passenger_schema;  
COMMENT ON SCHEMA passenger_schema IS 'Railway passenger ticket database schema';  
SET search_path TO passenger_schema;
```

```
CREATE TABLE passport (  
    id SERIAL PRIMARY KEY,  
    series VARCHAR(10) NOT NULL,  
    number VARCHAR(20) NOT NULL,
```

```

    issue_date DATE NOT NULL,
    CONSTRAINT uq_passport_series_number UNIQUE (series, number),
    CONSTRAINT chk_passport_issue_date CHECK (issue_date <= CURRENT_DATE),
    CONSTRAINT chk_passport_series_not_empty CHECK (length(trim(series)) > 0),
    CONSTRAINT chk_passport_number_not_empty CHECK (length(trim(number)) > 0)
);

```

```

CREATE TABLE passenger (
    id SERIAL PRIMARY KEY,
    id_passport INTEGER NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    middle_name VARCHAR(50),
    CONSTRAINT uq_passenger_passport UNIQUE (id_passport),
    CONSTRAINT chk_passenger_last_name CHECK (length(trim(last_name)) > 0),
    CONSTRAINT chk_passenger_first_name CHECK (length(trim(first_name)) > 0),
    CONSTRAINT fk_passenger_passport FOREIGN KEY (id_passport)
        REFERENCES passport(id) ON UPDATE CASCADE ON DELETE RESTRICT
);

```

```

CREATE TABLE station (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    city VARCHAR(100) NOT NULL,
    CONSTRAINT uq_station_name_city UNIQUE (name, city),
    CONSTRAINT chk_station_name CHECK (length(trim(name)) > 0),
    CONSTRAINT chk_station_city CHECK (length(trim(city)) > 0)
);

```

```

CREATE TABLE ticket_office (
    id SERIAL PRIMARY KEY,
    address VARCHAR(255) NOT NULL,
    number VARCHAR(20) NOT NULL,
    CONSTRAINT uq_ticket_office_number UNIQUE (number),
    CONSTRAINT chk_ticket_office_address CHECK (length(trim(address)) > 0),
    CONSTRAINT chk_ticket_office_number CHECK (length(trim(number)) > 0)
);

```

```

CREATE TABLE train (
    id SERIAL PRIMARY KEY,
    number VARCHAR(10) NOT NULL,
    type VARCHAR(20) NOT NULL,
    CONSTRAINT uq_train_number UNIQUE (number),
    CONSTRAINT chk_train_number CHECK (length(trim(number)) > 0),
    CONSTRAINT chk_train_type CHECK (type IN ('fast', 'passenger', 'branded', 'express'))
);

```

```

CREATE TABLE carriage (
    id SERIAL PRIMARY KEY,
    number VARCHAR(10) NOT NULL,
    type VARCHAR(20) NOT NULL,

```

```

        seats_count INTEGER NOT NULL,
        CONSTRAINT chk_carriage_number CHECK (length(trim(number)) > 0),
        CONSTRAINT chk_carriage_type CHECK (type IN ('compartment', 'reserved_seat', 'seated', 'luxury')),
        CONSTRAINT chk_carriage_seats_count CHECK (seats_count > 0)
    );

```

```

CREATE TABLE train_carriage (
    id_train INTEGER NOT NULL,
    id_carriage INTEGER NOT NULL,
    CONSTRAINT pk_train_carriage PRIMARY KEY (id_train, id_carriage),
    CONSTRAINT fk_train_carriage_train FOREIGN KEY (id_train)
        REFERENCES train(id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_train_carriage_carriage FOREIGN KEY (id_carriage)
        REFERENCES carriage(id) ON UPDATE CASCADE ON DELETE RESTRICT
);

```

```

CREATE TABLE route (
    id SERIAL PRIMARY KEY,
    number VARCHAR(20) NOT NULL,
    start_station INTEGER NOT NULL,
    end_station INTEGER NOT NULL,
    CONSTRAINT uq_route_number UNIQUE (number),
    CONSTRAINT chk_route_number CHECK (length(trim(number)) > 0),
    CONSTRAINT chk_route_stations_not_equal CHECK (start_station <> end_station),
    CONSTRAINT fk_route_start_station FOREIGN KEY (start_station)
        REFERENCES station(id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_route_end_station FOREIGN KEY (end_station)
        REFERENCES station(id) ON UPDATE CASCADE ON DELETE RESTRICT
);

```

```

CREATE TABLE route_station (
    id_route INTEGER NOT NULL,
    id_station INTEGER NOT NULL,
    station_order INTEGER NOT NULL,
    stop_duration INTEGER NOT NULL,
    distance_from_start INTEGER NOT NULL,
    CONSTRAINT pk_route_station PRIMARY KEY (id_route, id_station),
    CONSTRAINT uq_route_station_order UNIQUE (id_route, station_order),
    CONSTRAINT chk_route_station_order CHECK (station_order > 0),
    CONSTRAINT chk_route_station_stop_duration CHECK (stop_duration >= 0),
    CONSTRAINT chk_route_station_distance CHECK (distance_from_start >= 0),
    CONSTRAINT fk_route_station_route FOREIGN KEY (id_route)
        REFERENCES route(id) ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_route_station_station FOREIGN KEY (id_station)
        REFERENCES station(id) ON UPDATE CASCADE ON DELETE RESTRICT
);

```

```

CREATE TABLE trip (
    id SERIAL PRIMARY KEY,
    id_route INTEGER NOT NULL,
    id_train INTEGER NOT NULL,

```

```

departure_datetime TIMESTAMP NOT NULL,
arrival_datetime TIMESTAMP NOT NULL,
CONSTRAINT chk_trip_datetime CHECK (arrival_datetime > departure_datetime),
CONSTRAINT fk_trip_route FOREIGN KEY (id_route)
    REFERENCES route(id) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_trip_train FOREIGN KEY (id_train)
    REFERENCES train(id) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE ticket (
    id SERIAL PRIMARY KEY,
    id_station INTEGER NOT NULL,
    id_ticket_office INTEGER NOT NULL,
    id_passenger INTEGER NOT NULL,
    id_trip INTEGER NOT NULL,
    id_carriage INTEGER NOT NULL,
    departure_station INTEGER NOT NULL,
    arrival_station INTEGER NOT NULL,
    ticket_price DECIMAL(10,2) NOT NULL,
    sale_datetime TIMESTAMP NOT NULL,
    seat_number INTEGER NOT NULL,
    CONSTRAINT chk_ticket_price CHECK (ticket_price > 0),
    CONSTRAINT chk_ticket_seat_number CHECK (seat_number > 0),
    CONSTRAINT chk_ticket_stations_not_equal CHECK (departure_station <> arrival_station),
    CONSTRAINT uq_ticket_place UNIQUE (id_trip, id_carriage, seat_number),
    CONSTRAINT fk_ticket_station FOREIGN KEY (id_station)
        REFERENCES station(id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_ticket_office FOREIGN KEY (id_ticket_office)
        REFERENCES ticket_office(id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_ticket_passenger FOREIGN KEY (id_passenger)
        REFERENCES passenger(id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_ticket_trip FOREIGN KEY (id_trip)
        REFERENCES trip(id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_ticket_carriage FOREIGN KEY (id_carriage)
        REFERENCES carriage(id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_ticket_departure_station FOREIGN KEY (departure_station)
        REFERENCES station(id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_ticket_arrival_station FOREIGN KEY (arrival_station)
        REFERENCES station(id) ON UPDATE CASCADE ON DELETE RESTRICT
);

```

4.5. Заполнение таблиц рабочими данными

Для проверки работы базы данных таблицы были заполнены рабочими данными. Данные приведены на английском языке, чтобы содержимое базы соответствовало требованиям преподавателя.

```

INSERT INTO passport (series, number, issue_date) VALUES
('AB', '1234567', '2016-04-12'),
('AB', '7654321', '2017-09-25'),

```

```
('MP', '4455667', '2018-02-18'),  
('MP', '9988776', '2019-11-03'),  
('KH', '1122334', '2020-06-14'),  
('KH', '5566778', '2015-12-01'),  
('BM', '2244668', '2021-03-20'),  
('BM', '8866442', '2014-08-09'),  
('MC', '1357924', '2022-01-30'),  
('MC', '2468135', '2013-05-22');
```

```
INSERT INTO passenger (id_passport, last_name, first_name, middle_name) VALUES  
(1, 'Smith', 'John', 'Robert'),  
(2, 'Johnson', 'Emily', 'Grace'),  
(3, 'Brown', 'Michael', 'James'),  
(4, 'Taylor', 'Olivia', 'Anne'),  
(5, 'Anderson', 'Daniel', 'Paul'),  
(6, 'Thomas', 'Sophia', 'Marie'),  
(7, 'Jackson', 'William', 'Henry'),  
(8, 'White', 'Emma', 'Rose'),  
(9, 'Harris', 'Alexander', 'David'),  
(10, 'Martin', 'Charlotte', 'Jane');
```

```
INSERT INTO station (name, city) VALUES  
('Central Station', 'Minsk'),  
('East Station', 'Moscow'),  
('Main Station', 'Saint Petersburg'),  
('South Station', 'Gomel'),  
('Passenger Station', 'Brest'),  
('North Station', 'Vitebsk'),  
('Central Station', 'Grodno'),  
('Railway Station', 'Mogilev');
```

```
INSERT INTO ticket_office (address, number) VALUES  
('1 Railway Square, Minsk', 'OF-001'),  
('25 Independence Avenue, Minsk', 'OF-002'),  
('10 Sovetskaya Street, Gomel', 'OF-003'),  
('4 Ordzhonikidze Street, Brest', 'OF-004'),  
('7 Kirov Street, Vitebsk', 'OF-005');
```

```
INSERT INTO train (number, type) VALUES  
('101A', 'fast'),  
('202B', 'passenger'),  
('303C', 'branded'),  
('404D', 'express');
```

```
INSERT INTO carriage (number, type, seats_count) VALUES  
('01', 'compartment', 36),  
('02', 'reserved_seat', 54),  
('03', 'seated', 60),  
('04', 'luxury', 18),  
('05', 'compartment', 36),  
('06', 'reserved_seat', 54),
```

```
('07', 'seated', 60),  
('08', 'compartment', 36);
```

```
INSERT INTO train_carriage (id_train, id_carriage) VALUES  
(1, 1), (1, 2), (1, 4),  
(2, 3), (2, 6),  
(3, 5), (3, 8),  
(4, 7);
```

```
INSERT INTO route (number, start_station, end_station) VALUES  
('R-101', 1, 2),  
('R-202', 1, 3),  
('R-303', 4, 5),  
('R-404', 6, 7);
```

```
INSERT INTO route_station (id_route, id_station, station_order, stop_duration, distance_from_start) VALUES  
(1, 1, 1, 0, 0), (1, 8, 2, 10, 190), (1, 2, 3, 0, 720),  
(2, 1, 1, 0, 0), (2, 6, 2, 12, 280), (2, 3, 3, 0, 830),  
(3, 4, 1, 0, 0), (3, 1, 2, 15, 310), (3, 5, 3, 0, 650),  
(4, 6, 1, 0, 0), (4, 1, 2, 10, 280), (4, 7, 3, 0, 520);
```

```
INSERT INTO trip (id_route, id_train, departure_datetime, arrival_datetime) VALUES  
(1, 1, '2026-05-15 08:30:00', '2026-05-15 18:10:00'),  
(2, 3, '2026-05-16 09:00:00', '2026-05-16 21:40:00'),  
(3, 2, '2026-05-17 07:20:00', '2026-05-17 16:30:00'),  
(4, 4, '2026-05-18 12:15:00', '2026-05-18 19:55:00'),  
(1, 1, '2026-05-20 08:30:00', '2026-05-20 18:10:00');
```

```
INSERT INTO ticket (id_station, id_ticket_office, id_passenger, id_trip, id_carriage,  
departure_station, arrival_station, ticket_price, sale_datetime, seat_number) VALUES  
(1, 1, 1, 1, 1, 2, 95.50, '2026-05-01 10:15:00', 5),  
(1, 2, 2, 1, 2, 1, 2, 72.30, '2026-05-01 11:20:00', 18),  
(8, 1, 3, 1, 4, 8, 2, 64.00, '2026-05-02 09:40:00', 7),  
(1, 2, 4, 2, 5, 1, 3, 110.00, '2026-05-03 14:05:00', 11),  
(6, 5, 5, 2, 8, 6, 3, 80.75, '2026-05-04 16:30:00', 22),  
(4, 3, 6, 3, 3, 4, 5, 68.20, '2026-05-05 13:10:00', 31),  
(1, 1, 7, 3, 6, 1, 5, 54.90, '2026-05-06 12:00:00', 40),  
(6, 5, 8, 4, 7, 6, 7, 59.99, '2026-05-07 17:45:00', 12),  
(1, 2, 9, 5, 1, 1, 2, 96.00, '2026-05-08 10:10:00', 16),  
(8, 1, 10, 5, 2, 8, 2, 66.50, '2026-05-09 15:25:00', 28);
```

4.6. Проверка содержимого таблиц

После создания и заполнения таблиц выполнены запросы SELECT для проверки содержимого всех таблиц базы данных.

```
SELECT * FROM passport;  
SELECT * FROM passenger;  
SELECT * FROM station;  
SELECT * FROM ticket_office;  
SELECT * FROM train;
```

```

SELECT * FROM carriage;
SELECT * FROM train_carriage;
SELECT * FROM route;
SELECT * FROM route_station;
SELECT * FROM trip;
SELECT * FROM ticket;

```

	id [PK] integer	id_station integer	id_ticket_office integer	id_passenger integer	id_trip integer	id_carriage integer	departure_station integer	arrival_station integer	ticket_price numeric (10,2)	sale_datetime timestamp without time zone	seat_number integer
1	1	1	1	1	1	1	1	2	95.50	2026-05-01 10:15:00	5
2	2	1	2	2	1	2	1	2	72.30	2026-05-01 11:20:00	18
3	3	8	1	3	1	4	8	2	64.00	2026-05-02 09:40:00	7
4	4	1	2	4	2	5	1	3	110.00	2026-05-03 14:05:00	11
5	5	6	5	5	2	8	6	3	80.75	2026-05-04 16:30:00	22
6	6	4	3	6	3	3	4	5	68.20	2026-05-05 13:10:00	31
7	7	1	1	7	3	6	1	5	54.90	2026-05-06 12:00:00	40
8	8	6	5	8	4	7	6	7	59.99	2026-05-07 17:45:00	12
9	9	1	2	9	5	1	1	2	96.00	2026-05-08 10:10:00	16
10	10	8	1	10	5	2	8	2	66.50	2026-05-09 15:25:00	28

Рисунок 3 – Результат выполнения SELECT-запросов для проверки таблиц

4.7. Резервное копирование базы данных

Для базы данных были созданы две резервные копии: Passenger.backup в формате Custom для восстановления базы данных и Passenger.sql в формате Plain для просмотра SQL-листинга.

Custom backup:

Database Passenger -> Backup...

Filename: Passenger.backup

Format: Custom

Encoding: UTF8

Data Options: Pre-data, Data, Post-data

SQL listing backup:

Database Passenger -> Backup...

Filename: Passenger.sql

Format: Plain

Encoding: UTF8

Query Options: Use INSERT Commands = ON, Include CREATE DATABASE statement = ON

Data Options: Pre-data, Data, Post-data

Backup (Database: Passenger) ✕

General Data Options Query Options Table Options Options Objects

Filename C:\Users\didik\OneDrive\Desktop\dumps\Passenger.backup 📁

Format Custom | ▾

Compression ratio

Encoding UTF8 ✕ | ▾

Number of jobs

Role name Select an item... | ▾

ℹ ? ✕ Close ↺ Reset 💾 Backup

Рисунок 4 – Создание резервной копии *Passenger.backup* в формате *Custom*

Backup (Database: Passenger)

General Data Options Query Options Table Options Options Objects

Filename C:\Users\didik\OneDrive\Desktop\dumps\Passenger.sql

Format Plain

Compression ratio

Encoding UTF8

Number of jobs

Role name Select an item...

Close Reset Backup

Рисунок 5 – Создание резервной копии *Passenger.sql* в формате *Plain*

4.8. Восстановление базы данных

Для проверки корректности резервного копирования база данных может быть восстановлена в pgAdmin через команду Restore. Для восстановления используется файл *Passenger.backup*, созданный в формате Custom.

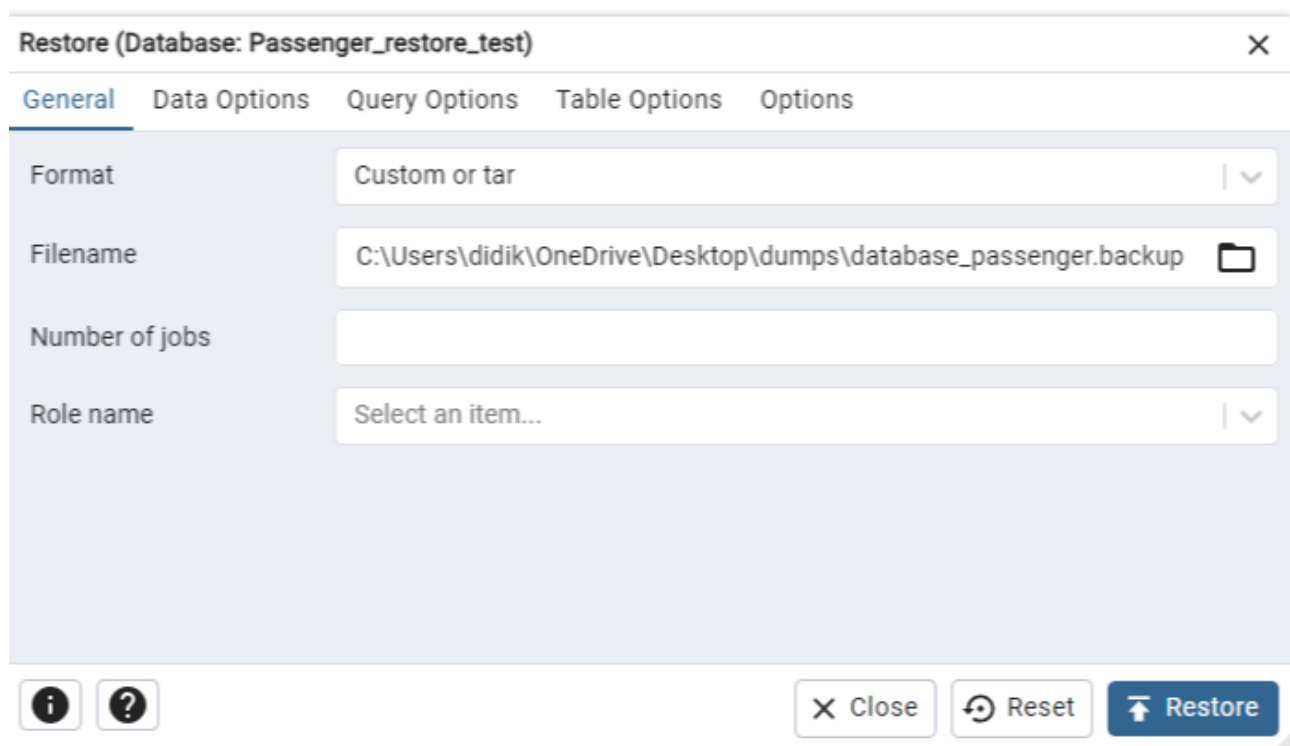


Рисунок 6 – Восстановление базы данных из файла *Passenger.backup*

5. Выводы

В ходе выполнения лабораторной работы была создана база данных Passenger в PostgreSQL. Были разработаны таблицы, соответствующие логической модели предметной области, заданы первичные и внешние ключи, ограничения уникальности и проверочные ограничения. Таблицы были заполнены рабочими данными, после чего была выполнена проверка содержимого таблиц с помощью SELECT-запросов. Также были созданы резервные копии базы данных в форматах Custom и Plain SQL для восстановления и просмотра SQL-листинга.